# Komorebi Preservation Protocol

## A Theory of Recoverable Research Continuity

*Preserve what existed. Record what happened. Revise what is believed. Make continuation explicit.*

**Author:** Komorebi P.
**Version:** Specification Draft 0.1
**Date:** September 2, 2026
**Scope:** All present and future Komorebi research projects

## Status of This Document

This document defines a forward-looking preservation theory and operating protocol for the Komorebi research program. It is a specification draft, not a claim that the described software infrastructure has already been implemented. Its purpose is to establish the normative structure that future implementations, archives, handoffs, and public releases should satisfy.

The protocol formalizes practices already present in parts of the research corpus, including versioned packages, sealed results, hash manifests, clean extraction tests, explicit failure records, and runnable Windows handoffs. It extends those practices into one project-independent theory of continuity.

**Scope boundary.** Preservation does not establish the truth, novelty, or importance of a result. It preserves the evidence, reasoning products, decisions, and operational state required to assess those questions honestly.

## Contents

**Status of This Document** i

**1 Executive Summary** 1

**2 Definition of Preservation** 1

2.1 Operational definition . . . 1
2.2 Formal state model . . . 2
2.3 Preservation equivalence . . . 2

**3 Preservation Axioms** 3

3.1 Identity conservation . . . 3
3.2 Monotone history and revisable belief . . . 3
3.3 Provenance closure . . . 3
3.4 Epistemic separation . . . 3
3.5 Canonicality . . . 3
3.6 Total outcomes . . . 3
3.7 Branch conservation . . . 3
3.8 Residual conservation . . . 4
3.9 Tested recoverability . . . 4
3.10 Controlled disclosure . . . 4

**4 The Continuity Block** 4

4.1 Block structure . . . 4

4.2 State-changing operators . . . 4

**5 Forward Operating Protocol 5**

5.1 Project charter . . . 5
5.2 Three canonical heads . . . 6
5.3 Begin . . . 6
5.4 Work and checkpoint . . . 6
5.5 Seal . . . 7

**6 Progressive Rigor and Promotion Gates 7**

6.1 Three preservation levels . . . 7
6.2 Promotion gates . . . 7

**7 Claims, Evidence, and Decisions 8**

7.1 Claim records . . . 8
7.2 Independent status axes . . . 8
7.3 Evidence graph . . . 9
7.4 Decision records . . . 9

**8 Failures and Residuals 9**

8.1 Failure taxonomy . . . 9
8.2 Residual ledger . . . 10

**9 AI and Tool Provenance 10**

**10 Project Validation Profiles 11**

**11 Retention, Recovery, and Integrity 11**

11.1 Optimal retention rule . . . 11
11.2 Integrity rules . . . 12
11.3 Recovery doctrine . . . 12

**12 Private, Internal, and Public Views 12**

**13 Forward Adoption Doctrine 12**

**14 Acceptance Tests and Health Criteria 13**

14.1 Cold-successor test . . . 13
14.2 Hard acceptance tests . . . 13

14.3 Health vector . . . . . 14

**15 Preservation Completeness** **14**

**16 Protocol Constitution** **14**

**A Minimal Continuity Capsule** **15**

**B Minimal Machine Record** **15**

**C Suggested Forward Commands** **16**

# 1 Executive Summary

The optimal preservation system is not merely a collection of files. It is a forward-only research state machine that preserves the ability to verify identity and integrity, understand, challenge, reproduce, restore, and continue every consequential state.

Here, recoverability means that operational state can be restored and canonical pointers can be rolled back. Recorded history itself remains append-only and is never reversed.

The Komorebi Preservation Protocol, abbreviated KPP, is built around one central law:

**History is irreversible. Belief is revisable.**

Earlier artifacts, runs, interpretations, and belief states remain recoverable. Current claims may nevertheless be strengthened, weakened, divided, refuted, restricted, superseded, or withdrawn. Correction therefore advances the record without rewriting it.

The protocol separates four kinds of truth:

Table 1: The four preserved truths.

| Truth | Question answered | Governing structure |
|---|---|---|
| **Artifact truth** | What exact statements, files, inputs, models, data, and outputs existed? | Immutable objects |
| **Process truth** | What derivation, command, experiment, review, failure, or decision occurred? | Append-only ledger |
| **Epistemic truth** | What does the evidence currently support, contradict, or leave unresolved? | Revisable claim graph |
| **Continuation truth** | Which state is authoritative, what remains open, and where should work resume? | Canonical heads and handoffs |

A project is preserved only when all four remain available and mutually linked.

# 2 Definition of Preservation

## 2.1 Operational definition

A research program is preserved when a future competent collaborator, using only the preserved record and declared external dependencies, can:

1. identify the exact materials that existed;
2. locate the authoritative current state;
3. reconstruct how and why that state arose;
4. distinguish observation from interpretation and belief;
5. reproduce or adjudicate pivotal results under a declared equivalence standard;

6. identify failures, gaps, limitations, and unresolved branches; and
7. resume work without inventing missing premises.

Byte retention alone is storage. Preservation requires *inferential continuity*: the future record must contain enough structure to recover both the operational state and the justified path into it.

## 2.2 Formal state model

At time $t$, define the preserved research state as

$$S_t = (O_t, L_t, G_t, H_t, P_t), \tag{1}$$

where:

- $O_t$ is the set of immutable, content-addressed artifacts;
- $L_t$ is the append-only ledger of events, runs, decisions, and revisions;
- $G_t$ is the typed graph of claims, evidence, counterevidence, dependencies, and provenance;
- $H_t$ is the set of movable canonical pointers; and
- $P_t$ is the set of validation policies, equivalence rules, and replay contracts.

The two monotonicity requirements are

$$O_t \subseteq O_{t+1}, \qquad L_t \preceq L_{t+1}, \tag{2}$$

where $\preceq$ means that $L_t$ is a prefix of the later event history. Artifacts and events accumulate. Claims and canonical pointers may change only through recorded transitions.

## 2.3 Preservation equivalence

A recovered state $\widehat{S}_t$ is preservation-equivalent to $S_t$, written

$$\widehat{S}_t \simeq_{\mathcal{E}} S_t, \tag{3}$$

when the recovered state preserves artifact identity, lineage, claim status, unresolved residuals, and pivotal behavior under a declared equivalence contract $\mathcal{E}$. The contract must specify one of the following, or a domain-specific refinement:

- byte-exact equality;
- logical or structural equality;
- numerical equality within a fixed tolerance;
- statistical equivalence under a fixed criterion;
- behavioral equivalence on a frozen test set; or
- semantic equivalence under an explicit review standard.

The equivalence rule must be declared before it is used as promotion evidence. Post hoc tolerances do not provide valid confirmatory support.

# 3 Preservation Axioms

## 3.1 Identity conservation

The same durable identifier always denotes the same content. Moving, renaming, repackaging, or mirroring an artifact does not alter its identity. Changed bytes create a new digest and therefore a new object or revision.

## 3.2 Monotone history and revisable belief

Events and evidence accumulate. Beliefs may evolve. A former claim state is never silently replaced by its successor. The successor links to it through a typed relation such as `corrects`, `supersedes`, `restricts`, `refutes`, or `withdraws`.

## 3.3 Provenance closure

Every consequential result must resolve to its exact inputs, assumptions, transformation, environment, producing actor or tool, and parent state. If a dependency is unavailable, the record must say so explicitly and bound the consequences.

## 3.4 Epistemic separation

Observations, interpretations, decisions, and claims are distinct record types. A software test does not prove a theorem. A clean replay does not establish external validity. A digest verifies byte identity, not scientific truth.

## 3.5 Canonicality

Every active project has exactly one authoritative head for each project-level role. Each named branch has exactly one branch head. Alternative states remain visible, but they cannot silently compete for the label `stable`.

## 3.6 Total outcomes

Every declared run or proof attempt receives a terminal disposition. Valid dispositions include success, null, mixed, refuted, invalid, blocked, crashed, cancelled, and abandoned. Unfavorable and incomplete outcomes do not disappear.

## 3.7 Branch conservation

Branching preserves the shared parent. Merging preserves all parents and records what was adopted, rejected, modified, or left unresolved. A merge must not fabricate a clean linear history.

### 3.8 Residual conservation

Gaps, anomalies, contradictions, failed controls, instability, regressions, and deferred questions are first-class records. They define the actual research frontier and therefore cannot be treated as archival noise.

### 3.9 Tested recoverability

Recoverability is established by successful restoration, not inferred from the existence of copies. Before promotion to `stable`, every milestone must complete a successful artifact restoration. Pivotal executable claims must also complete the applicable clean replay, while non-executable claims undergo their declared adjudication procedure.

### 3.10 Controlled disclosure

A lossless private master, a curated internal record, and a sanitized public release may share lineage but remain distinct views. Redaction never changes the private master. Public material is generated through an explicit allowlist and independent validation.

## 4 The Continuity Block

### 4.1 Block structure

Every meaningful unit of work creates a parent-linked transition

$$S_n \xrightarrow{B_n} S_{n+1}, \tag{4}$$

with continuity block

$$B_n = (P, I, X, T, O, E, R, D, N, M). \tag{5}$$

A block is mutable while open. Once sealed, it becomes immutable. Later corrections create amendment blocks linked to the original. A scientifically unsuccessful block can still be preservation-complete.

### 4.2 State-changing operators

The principal operators are:

$$\text{Revise}(c \to c') : \text{preserve } c\text{, add } c'\text{, and record cause and evidence,} \tag{6}$$
$$\text{Branch}(s \to s_1, \ldots, s_k) : \text{preserve the parent and assign branch hypotheses,} \tag{7}$$
$$\text{Merge}(s_1, \ldots, s_k \to m) : \text{record all parents, conflicts, and dispositions,} \tag{8}$$
$$\text{Fail}(r) : \text{preserve protocol, observation, logs, class, and claim impact,} \tag{9}$$
$$\text{Handoff}(h) : \text{close provenance, evidence, operation, and continuation around } h. \tag{10}$$

Promotion changes a pointer. It never destroys the states behind that pointer.

Table 2: Required components of a continuity block.

| Term | Component | Requirement |
|---|---|---|
| $P$ | **Parent** | Exact state or states from which the block begins |
| $I$ | **Intent** | Bounded objective, question, or hypothesis |
| $X$ | **Inputs** | Artifacts, assumptions, models, datasets, and dependencies |
| $T$ | **Transformation** | Derivation, code change, command, intervention, or review |
| $O$ | **Outputs** | Every consequential produced artifact |
| $E$ | **Evidence** | Support, counterevidence, null evidence, or limitations produced |
| $R$ | **Residuals** | Gaps, anomalies, failures, instability, and deferred questions |
| $D$ | **Decision** | Claim changes, branch disposition, or engineering choice |
| $N$ | **Next** | Clearest admissible continuation point |
| $M$ | **Manifest** | Stable identifiers, digests, lineage, and replay contract |

# 5 Forward Operating Protocol

## 5.1 Project charter

Every new project begins with a compact charter containing:

- stable project identifier and title;
- objective, scope, and exclusions;
- current `work`, `stable`, and `public` heads;
- claim and run types used by the project;
- project-specific validation profile;
- data, privacy, and publication policy;
- primary environment and external dependencies;
- definition of `stable`; and
- definition of `public-ready`.

The charter should take minutes to establish. It changes only through an explicit charter revision.

### 5.2 Three canonical heads

Each project maintains:

| | |
|---|---|
| `work` | a project-level manifest naming the default active state and every live branch head; |
| `stable` | the newest state that has passed its declared stability gates; |
| `public` | the newest sanitized public release. |

A rollback repoints a head to an earlier preserved state. It does not erase later work.

### 5.3 Begin

Before a meaningful work block, record four judgments:

1. the bounded objective;
2. the success or falsification criterion;
3. whether the work is exploratory or confirmatory; and
4. the expected consequential output.

The system should automatically resolve and verify the parent state, assign a block identifier, capture known local changes, identify exact inputs, and record the initial visibility classification.

### 5.4 Work and checkpoint

Normal research should continue with minimal ceremony. Evidential commands should automatically capture:

- exact command and working directory, with secrets redacted;
- source state or content digest;
- dependency, runtime, and hardware fingerprint;
- inputs and outputs by digest;
- random seeds and trial identity;
- start time, finish time, and exit code;
- standard output and error logs after credential and sensitive-output filtering; and
- deviations from the declared protocol.

Lightweight checkpoints should be created before dependency changes, before risky refactors, after pivotal results, at interruption, and whenever the cost of reconstructing work since the last checkpoint becomes material.

### 5.5 Seal

At closure, automatic tooling should inventory changes, validate manifests, account for all declared runs, execute configured tests, and create the continuity packet. The canonical closure capsule contains six short judgments:

1. purpose;
2. outcome;
3. uncertainty;
4. claim impact;
5. decision; and
6. next action.

Purpose is inherited from the opening record unless the objective changed. Outcome may be drafted from captured runs, but a human must confirm its interpretation. The remaining fields record judgments that cannot be reconstructed reliably from files alone.

The protocol should fail open during exploration and fail closed during promotion. Missing metadata may produce an incomplete private checkpoint. It cannot produce a stable or public release.

## 6 Progressive Rigor and Promotion Gates

### 6.1 Three preservation levels

Table 3: Progressive rigor prevents archival ceremony from displacing research.

| Level | Automatic content | Human judgment |
|---|---|---|
| **Automatic capture** | Identity, parent, file hashes, commands, inputs, outputs, logs, environment, exit states | None or minimal classification |
| **Continuity capsule** | Links to captured objects and current state | Purpose, outcome, uncertainty, claim impact, decision, next action |
| **Certified checkpoint** | Frozen snapshot, manifest, replay recipe, complete validation, redundant copies | Claim audit, limitations, promotion decision |

Anything mechanically derivable should be generated automatically. No person should enter the same fact twice. Full ceremony is reserved for consequential claims and releases.

### 6.2 Promotion gates

Table 4: Forward promotion gates.

| Gate | Name | Requirement |
|---|---|---|
| **G0** | **Captured** | Stable identifiers, parent, timestamp, provenance, visibility, and content digests assigned |
| **G1** | **Sealed** | Manifest valid, planned runs accounted for, and no silent overwrites |
| **G2** | **Evidentially complete** | Protocol, raw outcomes, interpretation, claim impact, counterevidence, and residuals recorded |
| **G3** | **Verified** | Project tests and declared comparison rules pass, with failures classified |
| **G4** | **Restored and replayed** | Artifact restoration succeeds, and each pivotal executable result reproduces from a clean directory under its declared equivalence rule |
| **G5** | **Stable** | Regressions pass, claims are updated, decision is recorded, and the stable head is intentionally advanced |
| **G6** | **Handoff-ready** | A cold successor can identify the state, reproduce a pivotal check, understand limitations, and begin the next block |
| **G7** | **Public-ready** | Allowlist export, privacy scan, license review, restricted-lineage review, and clean public replay pass |

A failed scientific hypothesis may reach G2 or higher as a preserved negative result. It cannot be misrepresented as a positive result merely because the package itself verifies successfully.

# 7 Claims, Evidence, and Decisions

## 7.1 Claim records

Every consequential claim receives a permanent identifier and revision history. Its record contains:

- exact formal and plain-language statements;
- claim type, scope, quantifiers, and assumptions;
- supporting and opposing evidence;
- proof or protocol dependencies;
- falsification criteria;
- boundary cases and known limitations;
- current revision and superseded revisions; and
- the event responsible for every status transition.

## 7.2 Independent status axes

A single confidence number may supplement these fields but may not replace them.

Table 5: Claim status is multidimensional.

| Axis | Example values |
|---|---|
| **Nature** | definition, lemma, theorem, conjecture, empirical, causal, engineering, interpretive |
| **Evidence** | untested, supported, mixed, contradicted, refuted, inconclusive |
| **Proof** | absent, incomplete, checked, independently checked, formally verified |
| **Reproduction** | unreplayed, clean local replay, fresh environment, independent implementation |
| **Lifecycle** | active, restricted, superseded, withdrawn |
| **Novelty conclusion** | unassessed, known precedent, candidate novel |
| **Novelty review** | not reviewed, internal review, independent review |
| **Visibility** | private, internal, public-candidate, public |

### 7.3 Evidence graph

The canonical evidential chain is

$$\text{claim} \longrightarrow \text{assumptions} \longrightarrow \text{derivation or implementation} \longrightarrow \text{protocol} \longrightarrow \text{run} \longrightarrow \text{raw result} \longrightarrow \text{interpretation} \longrightarrow \text{decision} \longrightarrow \text{release}.$$

Missing links are permitted only when explicitly marked. The graph must support both forward lineage and reverse impact analysis. For example, invalidating one run should reveal every figure, interpretation, claim, and release that depends upon it.

### 7.4 Decision records

A consequential decision records the decision made, evidence considered, alternatives rejected, rationale, reversibility, and conditions that should trigger reconsideration. Preserving files without preserving decision logic produces operational memory without judgment.

## 8 Failures and Residuals

### 8.1 Failure taxonomy

A dependency conflict is not scientific counterevidence. A valid negative result is not an engineering inconvenience. Every planned seed and trial must receive an outcome disposition, preventing accidental success-only reporting.

Table 6: Failure classes must not be conflated.

| Class | Meaning |
|---|---|
| **Scientific refutation** | A valid result contradicts the tested claim |
| **Scientific null** | A valid test finds no expected effect |
| **Mixed result** | Support varies across conditions, seeds, models, tasks, or scales |
| **Engineering failure** | Dependency, memory, file, implementation, or runtime failure |
| **Protocol invalid** | The attempted test cannot answer its intended question |
| **Inconclusive** | Power, coverage, controls, or precision are inadequate |
| **Abandoned branch** | Work stopped for a recorded reason without resolution |

### 8.2 Residual ledger

Closing a block requires disposition, not resolution. Every residual is marked `open`, `resolved`, `accepted`, `deferred`, or `invalidated`, together with its next trigger or owner. The residual ledger is the map of the living frontier.

## 9 AI and Tool Provenance

Attribution and validation must be recorded separately. For every material model-assisted contribution, preserve:

- model or system identifier exactly as exposed, or `unknown`;
- date and visible session or task reference;
- role, such as proposed, derived, coded, reviewed, executed, or interpreted;
- files, claims, experiments, or decisions affected;
- human disposition, such as accepted, edited, rejected, or unreviewed; and
- subsequent verification and verifier.

Visible prompts and outputs may be retained when relevant, or represented by a digest and private transcript reference. Hidden internal reasoning is neither required nor claimed. A model reviewing its own proposal is an internal consistency check, not independent replication.

**Illustrative provenance statement.** A human collaborator supplies an ambient-space hypothesis. A model develops an operator formulation and implementation draft. The human selects the design. Automated tests are executed locally. The result remains independently unreplicated.

The archive must be agent-independent. A new capable human or model should be able to resume from the internal or handoff continuity state without requiring undocumented memory from an earlier interaction. Public-only continuation is promised only when the public release declares and supplies sufficient dependencies.

# 10 Project Validation Profiles

The universal envelope remains small. Each domain supplies its own validation profile.

Table 7: Project-specific promotion requirements.

| Project family | Required emphasis |
|---|---|
| **Komorebi Math** | Exact statements, assumption closure, theorem dependency graph, boundary cases, and explicit separation of numerical evidence from proof |
| **Neuralese** | Interventions, null and shuffled controls, multiple seeds and layers, OOD behavior, and depth stability |
| **RRCT** | Operational and causal results separated from interpretations concerning awareness or subjectivity |
| **OSL** | Grammar, formal semantics, conformance vectors, round-trip tests, version negotiation, and interpreter equivalence |
| **ARC** | Frozen task release and split, solver digest, scorer, leakage audit, compute budget, and per-task outputs |
| **SRIC** | Frozen pre-improvement baseline, human changes separated from autonomous changes, causal attribution, compute-normalized gains, regressions, and rollback |
| **WaveKAT-F** | Type I error calibration, complete simulation seeds, comparator parity, GPU and reference agreement, and cohort provenance |

These profiles may evolve through versioned policy changes. A result is certified against the exact policy revision in force at promotion time.

# 11 Retention, Recovery, and Integrity

## 11.1 Optimal retention rule

Preserve losslessly and redundantly anything that is:

- irreplaceable;
- raw evidence;
- decision-bearing;
- expensive to regenerate;
- necessary for interpretation; or

- necessary for reproduction.

Derived caches may be discarded only when exact parent objects and a tested regeneration procedure survive. Duplicate bytes should have one logical content identity within each repository or replica set and be referenced by digest. That logical object must still have multiple physically independent replicas. Environments and large model weights should normally be referenced by exact digest and legal acquisition method unless complete offline preservation is required and permitted.

### 11.2 Integrity rules

Each artifact records an algorithm-tagged content digest, byte length, media type, stable identifier, provenance, and role. Each milestone records a deterministic logical manifest. A content digest proves byte identity only. If signatures are later introduced, a signature attributes a statement to a key but still does not establish scientific truth.

### 11.3 Recovery doctrine

Each stable state should have multiple independently controlled copies, automatic integrity checks, encrypted private copies where appropriate, separately protected recovery credentials, open metadata formats, periodic random replay, and a full cold-restoration drill at major milestones.

Recovery should restore into a new location by default. It must not silently overwrite the active workspace. A restoration event becomes part of the append-only ledger.

## 12 Private, Internal, and Public Views

The protocol preserves three separate surfaces:

1. a lossless private master;
2. a curated internal research record; and
3. a sanitized public release.

Public packages are generated from an explicit allowlist in a clean location. They are not created by copying the private tree and deleting obvious secrets. Scanning must cover source files, logs, stack traces, notebook outputs, filenames, paths, metadata, manifests, and version-control history.

Credentials and live secrets are never archived as ordinary content. They are excluded or redacted at ingestion and represented only by references to a separately controlled secret-management mechanism. Private by default are personal conversations, machine-identifying paths, restricted datasets, licensed weights, private reviewer material, necessary raw model interactions, and unaudited historical packages. Publication should never mutate or replace the private source record.

## 13 Forward Adoption Doctrine

The protocol begins prospectively. Research should not pause for a comprehensive migration of the historical corpus.

1. Establish one genesis checkpoint for every currently active project.

2. Identify its current `work`, `stable`, and `public` heads.
3. Apply KPP to every consequential block from that point forward.
4. Introduce older material only when it is reused, validated, cited, or prepared for release.
5. Mark older material as an inherited input with known provenance, verification state, uncertainties, and dependent claims.

This approach creates immediate continuity without allowing archival normalization to consume the research program.

# 14 Acceptance Tests and Health Criteria

## 14.1 Cold-successor test

A stable state is handoff-complete only when another competent human or agent can, without relying on the original conversation:

- identify the authoritative state;
- explain why it became authoritative;
- locate supporting and opposing evidence;
- reproduce one pivotal result under its declared equivalence rule;
- see every consequential unresolved residual; and
- begin the next block without inventing missing premises.

## 14.2 Hard acceptance tests

The protocol should be tested against at least the following cases:

1. Two differently named packages contain identical bytes. One logical object identity is created, multiple physical replicas may remain, and two intake events stay visible.
2. A theorem statement changes. The old statement remains fixed and the new revision links to it explicitly.
3. A run reports success but lacks exact inputs. It may be captured privately but cannot be promoted.
4. A public package contains a credential inside a log. Publication fails closed.
5. A result is refuted. Current status changes, while historical releases remain reproducible and visibly superseded.
6. A schema changes. Old records remain readable and migration is reversible and auditable.
7. One project reuses another project's component. It pins the shared identity rather than creating an ambiguous copy.
8. Archive storage is lost. A restoration drill reconstructs indexes and verifies artifact identities.
9. The original collaborator is unavailable. A successor identifies the canonical state and resumes from the handoff alone.

### 14.3 Health vector

Preservation quality should be reported as a vector, not collapsed into one score. Critical failure in security or recoverability must not be hidden by excellent hashing or extensive metadata.

Recommended dimensions include:

- canonical-head uniqueness;
- ancestry and provenance completeness;
- claim-to-evidence coverage;
- declared-run outcome coverage;
- replay success by equivalence class;
- unresolved orphan artifacts;
- restoration success;
- dependency availability;
- public-release scan status; and
- cold-successor handoff success.

## 15 Preservation Completeness

**Preservation completeness proposition.**
Relative to declared external dependencies and surviving storage substrates, a promoted project state is preservation-complete when it has immutable artifact closure, an append-only transition history, referentially closed provenance, a current multidimensional claim state, explicit residuals, a unique canonical head, a declared replay equivalence rule, at least one successful artifact restoration, and the applicable replay or adjudication procedure.

Removing any one component produces a characteristic ambiguity: unknown bytes, unknown origin, unknown evolution, unknown authority, unknown epistemic status, unknown frontier, or untested recovery.

The protocol therefore protects more than files. It preserves the program's operational and epistemic continuity: what existed, what happened, what was believed, why belief changed, what remains unresolved, and where work should continue.

## 16 Protocol Constitution

The following rules govern all future refinements of KPP:

1. Preserve enough to restore, roll back, explain, reproduce, and continue every consequential change without reversing recorded history.
2. Never strengthen a scientific claim merely because its package verifies successfully.
3. Never erase valid counterevidence, failed trials, or earlier rational belief states.
4. Never let two states silently claim the same canonical role.

5. Never make public disclosure the mechanism by which private preservation is achieved.
6. Never require archival ceremony whose value is lower than its risk of abandonment.
7. Generate mechanically derivable metadata automatically.
8. Permit explicit unknowns rather than fabricated precision.
9. Treat restoration as an experiment that must actually be run.
10. Keep the protocol itself versioned, inspectable, and correctable.

**The archive is not a museum of conclusions. It is the continuity substrate of a living, self-correcting research program.**

## A Minimal Continuity Capsule

The minimum human-readable closure record for an ordinary block is:

| | |
|---|---|
| **Purpose** | What were we trying to learn, establish, repair, or change? |
| **Outcome** | What actually happened, including failures and null results? |
| **Uncertainty** | What remains broken, unverified, ambiguous, or dependent on unavailable evidence? |
| **Claim impact** | Which claims were strengthened, weakened, split, restricted, or refuted? |
| **Decision** | What was adopted, rejected, deferred, or reversed? |
| **Next** | What is the clearest admissible continuation point? |

All remaining fields should be generated from the captured state wherever possible.

## B Minimal Machine Record

The irreducible machine-readable envelope is conceptually equivalent to:

```
record_id: permanent identifier
record_type: project | block | checkpoint | claim | evidence
             artifact | run | residual | decision | event
             policy | amendment | release | extension:<name>
project_id: permanent project identifier
schema_version: metadata schema
created_at: UTC timestamp
creator: human | model | tool, with identifier when known
parent_ids: direct ancestry
visibility: private | internal | public-candidate | public
content_digest: algorithm-tagged digest
```

```
purpose: short human-readable statement
relations: typed links to other records
```

Domain modules extend this envelope. They do not replace it.

## C Suggested Forward Commands

The eventual Windows-facing interface should remain compact:

The following commands are normative interface sketches. They do not assert that a `kpp` implementation already exists.

```
kpp begin neuralese "Test ambient-space stabilization"
kpp run -- python experiment.py --seed 17
kpp checkpoint
kpp seal
kpp verify neuralese@stable
kpp replay neuralese@stable --clean --offline
kpp handoff neuralese@stable --self-contained
kpp recover neuralese@stable
```